

Objectifs :

- Apprendre la programmation multithreading.
- Savoir reconnaître les données partagées entre différents threads.
- Être capable de gérer la synchronisation de threads au moyen des primitives de l'exclusion mutuelle.

Exercice 1

On veut écrire un programme qui recherche le maximum dans un tableau de NE entiers dont les valeurs varient de 0 à NE*100. NE, le nombre d'éléments du tableau ainsi que NT, le nombre de threads utilisés pour la recherche du maximum sont des paramètres du programme. Le thread principal découpe le tableau en NT parties et lance NT threads fils qui effectuent chacun la recherche dans leur partie du tableau. À la fin de sa recherche, chaque thread chercheur met à jour le maximum qui est une variable globale partagée par tous les threads. Le thread principal attend que tous les autres threads aient terminé avant d'afficher la valeur de la variable globale.

- Ecrire un programme en c permettant de rechercher le maximum dans un tableau.
- Écrire et tester la création et la terminaison des NT threads par le thread principal.
- Ajouter à la version précédente la recherche dans une sous-partie du tableau du maximum par chaque thread avec l'affichage de la valeur trouvée.
- Compléter ce programme pour obtenir une première version qui utilise un mutex pour protéger le maximum lors de sa mise à jour par chacun des threads chercheurs.

T1	T2	T1	T2	T1	T2	T1	T2
5	5	7	8	9	3	2	1
i=0	i=1	i=2	i=3	i=4	i=4	i=5	i=6

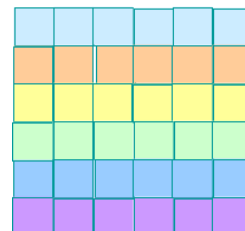
Exemple avec deux thread T1 et T2

Exercice 2

Ecrire un programme permettant d'effectuer la multiplication de deux matrices. Calculer le temps d'exécution avec un et plusieurs threads.

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} =$$

$$\begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} & a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} & a_{21}b_{13} + a_{22}b_{23} + a_{23}b_{33} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} & a_{31}b_{12} + a_{32}b_{22} + a_{33}b_{32} & a_{31}b_{13} + a_{32}b_{23} + a_{33}b_{33} \end{pmatrix}$$



Thread 0
Thread 1
Thread 2
Thread 3
Thread 4
Thread 5

```

double A[N][N], B[N][N], C[N][N];
int i, j, k;
for(i = 0; i < N; i++)
{
    for(j = 0; j < N; j++)
    {
        double sum = 0.0;
        for(k = 0; k < N; k++)
            sum += A[i][k] + B[k][j];
        C[i][j] = sum;
    }
}

```

Exercice 3

Ecrire un programme permettant de simuler la gestion du stock d'un magasin. Dans ce programme, il faut un seul thread pour la gestion du stock et N threads pour simuler les actions des clients. Les threads clients prennent dans le stock et le thread du magasin contrôle le stock (ajouter des items stock dès qu'il devient trop bas pour satisfaire les clients). Le nombre d'articles pris du stock sont des nombres aléatoires ainsi que l'ordre de passage des clients.

La gestion du stock d'un magasin est un exemple classique de programmation multithread avec mémoire partagée, qui simule un magasin (le producteur) et plusieurs clients (les consommateurs) à l'aide de la bibliothèque Pthreads. Le code donné (Stock.c) ne contient pas encore de mécanismes de synchronisation (comme les mutex ou les variables de condition). Ainsi, plusieurs threads peuvent accéder simultanément à la variable `store.valstock`, ce qui peut provoquer des conditions de concurrence (race conditions) et des incohérences dans le stock.

- Compilez et exécutez ce programme pour analyser le rôle de chaque partie du code et observer son comportement en l'absence de toute synchronisation.
- Compléter ce programme pour obtenir une version synchronisée (type moniteur) utilisant des mutex et variables de condition.